Shape the Line, Not the Paragraph

Drawing text line by line from a certified width model: the same pixels, exact positions, and a cost that follows what changes

Raouf Rahiche · Version 1.0 · 2026-11-05 · github.com/Rahiche/break-before-you-shape

A companion to *Break Before You Shape* (`paper/paper.html`). Every number here is read from result files produced on one Apple M3 machine and an Android emulator on it.

Abstract. Flutter shapes a paragraph as one long line, cuts that line into lines, and paints every line. A certified per-font width model (the sidecar of the companion paper) gives Flutter's exact line breaks before anything is shaped. We test what that allows at drawing time: laying out and drawing each line as its own one-line paragraph, at the position the model gives. In Flutter 3.47.5 on macOS, drawing line by line gave the same pixels as Flutter's own paragraph in 104,600 of 105,775 comparisons (98.9%), and at 16 px in every comparison under all three renderers we tried (Impeller and Skia on macOS, Impeller on an Android emulator). Where they differ, the paragraph is the less exact of the two: at 18.4 px a line drawn alone is within 0.00007 px of the exact positions, the same line in its paragraph up to 0.0021 px away. In long text the paragraph's coordinates move line breaks: in a 200,133-character text Flutter breaks 21 paragraphs differently from the same paragraphs laid out alone, and past a million pixels along its shaped line it starts new lines up to 65.7 px before a 330 px box is full. Drawn line by line, all 748 paragraphs match Flutter's layout of the paragraph alone. The cost then follows what changes: a streamed answer costs 26–42 μs per token at its end, from 651 to 8,241 characters, where one `Text` costs 1,509 μs per token at 8,241; and the first frame of a 200,133-character text takes 13.3 ms instead of 47.7 ms.

---

1 · Introduction

Read in its source (the companion repository's `research/ENGINE.md`, §2), Flutter's text engine, SkParagraph, does three things in order. `ParagraphImpl::layout` shapes the whole paragraph as one endless line (`shapeTextIntoEndlessLine`). `breakShapedTextIntoLines` then cuts that line into lines by summing cluster widths. `ParagraphImpl::paint` paints every line, whether it is on screen or not. Three consequences follow, and they are the subject of this paper.

- **Cost follows the paragraph, not the change.** Any change to the text makes a new paragraph, and Flutter's paragraph cache is keyed by the whole text: a streamed answer is shaped again, whole, at every token.
- **Cost follows the paragraph, not the screen.** A long text is shaped and laid out in full before its first frame, and every line is painted.
- **Positions come from one long line.** A line's glyphs are placed from the paragraph-wide shaping, sliced where the line starts. Skia's coordinates are float32 (`SkScalar`), which have coarse steps far along a long paragraph; §5–6 measure what that does.

The companion paper builds a per-font width model, distilled from Flutter itself and certified from the font's tables, that gives Flutter's line breaks and heights before any shaping. Once the breaks are known, nothing forces the engine to shape a paragraph as one line: each line can be laid out as its own one-line paragraph and drawn where the model says. This paper measures what that changes.

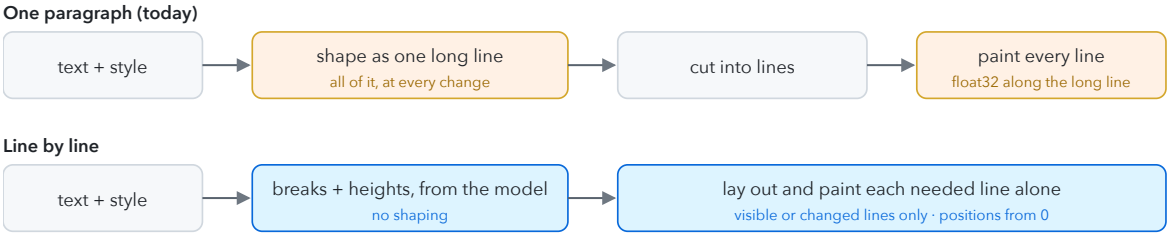


Figure 1. Today a paragraph is shaped as one long line and cut into lines. With the breaks known first, each line is laid out alone, and only the lines that are needed.

Contributions.

1. **Line-by-line drawing** for Flutter, from the sidecar's breaks: per-paragraph hand-backs, a streaming variant that re-breaks only the end of a growing text, and a long-text variant that lays out only the viewport.
2. **The same pixels.** A byte-for-byte comparison with Flutter's own paragraph across three renderers, four sizes, three alignments and three pixel ratios, with a control that proves the comparison can see a 1/8 px shift.
3. **A precision finding.** Where the two differ, the paragraph is the one that is off: its glyph positions drift with their distance along the shaped line, and in long text Flutter's own line breaks degrade with it, to lines started tens of pixels early.
4. **Costs.** Streaming and long-text timings against one `Text` and against one `Text` per paragraph.

---

2 · Drawing line by line

2.1 · Lines, and what goes back to Flutter

For a text, a font size and a width, the sidecar returns the start of every line, or hands the text back when it cannot be sure (a word as wide as the line, a fit decided within float error, a character it does not certify). A long text is split at its hard breaks, and each paragraph is decided on its own, with its hard break kept: on Apple platforms the character before a hard break kerns with it (measured in the companion paper). A paragraph handed back is laid out by Flutter as usual, and only that paragraph.

2.2 · Drawing

Line i's text runs from its start to the next line's start, with its trailing spaces and without the hard break that ends it. It is laid out by Flutter as a paragraph of its own, in the same style, width and alignment, and painted at $y = i \times h$, where h is the model's line height (Flutter's heights are lines $\times$ h for one font: measured, companion paper §4). The line alone is shaped from position 0. For certified text the companion paper shows a line's width is the sum of its characters' advances and in-line pair adjustments, so shaping it alone gives it the same advances; §4 checks the pixels.

2.3 · Streaming

A streamed answer grows at its end. Under greedy breaking, appending text can only move the breaks of the last lines: a line ends at the last opportunity that fits, and every earlier line's fit is decided by words that are already complete. The streaming variant therefore re-breaks the text from the start of its second-to-last line at each token (records for that tail only, then the greedy pass), lays out the lines whose text changed, and keeps every line before the second-to-last as final. Final lines are recorded into a picture, 16 at a time, so a token paints the picture and the last lines. If the sidecar hands the tail back, Flutter lays out the whole text, as it does today. §7.1 checks this against Flutter at every token.

2.4 · Long text

For a long text the sidecar gives every line start and the exact total height (arithmetic over the width prefix, plus Flutter's layout of any paragraph handed back), and only the lines in the viewport are laid out and painted. A scroll lays out the lines that come into view.

3 · Setup

Machine. Apple M3, macOS 26.3 (25D5087f); Flutter 3.47.5; release builds. Android: an emulator (sdk_gphone64_arm64, API 35, arm64-v8a) on the same machine; there is no phone in this study.

Renderers. macOS with Impeller on Metal (Flutter's default here), macOS with Skia on Metal (Impeller turned off in the built app), and the Android emulator with Impeller on OpenGL ES. Each run logs the backend it used.

Texts. The companion paper's corpora, built from this machine's files: Python docstrings, man pages, licence texts, and macOS interface strings in English, German, French, Russian, Ukrainian and Greek. Roboto Regular, with the companion paper's sidecar for it.

Pixels. For each text, size (px), width (150, 240 and 330 px) and configuration (start, centre and end alignment at 1×, start alignment at 2× and 3×), the paragraph and its lines are each drawn into a picture, rasterized with `Picture.toImage`, and compared byte for byte (RGBA). Control: in every start-aligned 1× case the lines are drawn again 1/8 px to the right, and that image must differ from the paragraph's.

Timing. The UI thread's work: laying out, and recording the paint into a picture. Rasterization runs on another thread and is not measured. Medians of 5 passes (streaming) or 7 (long text); every timed layout misses Flutter's paragraph cache.

4 · The same pixels

Renderer	Comparisons	Identical	Share	Differ at 16 px	Control seen
macOS, Impeller (Metal)	105,775	104,600	98.89%	0	21,155 of 21,155
macOS, Skia (Metal)	39,530	38,994	98.64%	0	7,906 of 7,906
Android emulator, Impeller (OpenGL ES)	39,500	39,130	99.06%	0	7,900 of 7,900

Table 1. Line-by-line drawing against Flutter's paragraph, byte for byte. Sizes: 12, 14, 16, 18.4 px on macOS Impeller (200 texts per corpus), 14, 16, 18.4 px under Skia and on the emulator (100 per corpus). In every comparison, the sidecar's line starts and heights equal Flutter's, and no line wraps when laid out alone.

macOS, Impeller	12 px	14 px	16 px	18.4 px
Comparisons	26,765	26,630	26,540	25,840
Identical	26,762	26,606	26,540	24,692
Where a differing comparison's first differing line starts (px along the shaped line)	beyond 8,192	beyond 4,096	none differ	under 2,048

Table 2. By size, macOS Impeller. The last row: the distance along the paragraph's one shaped line at which the first differing line of every differing comparison starts, in steps of 2,048, 4,096 and 8,192 px.

At 16 px every comparison is identical, under all three renderers. At the other sizes a small share differs; where it does, the largest difference in any channel of any pixel is 100 of 255 on macOS.

5 · Where they differ, the paragraph is the one that is off

We measured characters' left edges in both layouts (`getBoxesForRange`, every fourth character of each line and its last) and compared them with the exact positions: the model's width prefix, in doubles. The texts are 90 paragraphs of over 600 characters, 150 px wide.

	12 px	14 px	16 px	18.4 px
Lines measured	3,024	3,490	3,132	2,086
Farthest line start (px along the shaped line)	12,883	10,638	17,221	11,776

Largest error, in its paragraph (px)	0.00000	0.00000	0.00000	0.00215
Largest error, drawn alone (px)	0.00000	0.00000	0.00000	0.00007

Table 3. Reported character positions against the exact ones.

At 12, 14 and 16 px both layouts report exact positions on every line. At 18.4 px a line drawn alone stays within 0.00007 px of exact (Flutter's advances are fixed point, 1/65536 px), while the same line in its paragraph drifts with its distance along the shaped line, up to 0.0021 px, in steps that rise with float32's resolution at that distance (Figure 2).

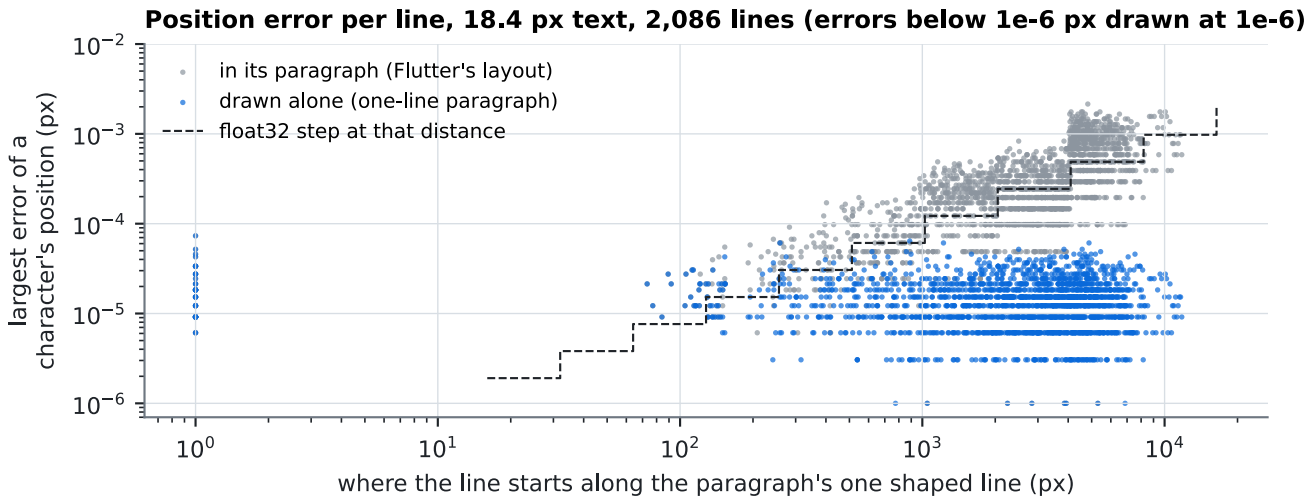


Figure 2. Each line's largest position error against where it starts along the paragraph's one shaped line, at 18.4 px. In its paragraph, the error grows with the distance. Drawn alone, the line starts at 0, and its error stays at most 0.00007 px.

The pixel differences have the same geography (Table 2). At 18.4 px they occur at every distance. At 14 px every differing comparison's first differing line starts beyond 4,096 px along the shaped line, at 12 px beyond 8,192 px, and at 16 px there are none. At 12 and 14 px the reported positions are exact in both layouts, so those differences arise when the glyphs are drawn far along the line; we have not traced them inside the renderers. The pattern is consistent with float32 coordinates measured from the start of the shaped line: a float32 near x moves in steps of $2^{\lfloor \log_2 x \rfloor - 23}$, 1/2048 px beyond 4,096 px and 1/1024 px beyond 8,192 px, and at 16 px Roboto's font units scale by 2^{-7} px, a power of two. A line drawn alone has its coordinates start at 0 and never goes far.

6 · Long text: the rounding moves Flutter's own line breaks

In a long `Text` the distances along the shaped line pass 100,000 px, where a float32 step is 1/128 px or more. The measurements below show that Flutter's line breaking sees them: a paragraph wraps differently depending on how much text comes before it.

Text	Lines	Paragraphs	Handed back	Same as Flutter, paragraph alone	Whole text breaks differently
10,437 characters	257	27	0	27	0
51,110 characters	1,284	143	5	143	1
200,133 characters	4,956	578	15	578	21

Table 4. Line breaks of long texts (paragraphs joined by hard breaks; 16 px, 330 px wide). "Same as Flutter, paragraph alone": the line-by-line breaks equal Flutter's layout of that paragraph on its own. "Whole text breaks differently": Flutter's layout of the whole text breaks that paragraph otherwise.

In each of the 21 paragraphs that the 200,133-character text breaks differently, Flutter laying out the paragraph alone agrees with the sidecar. We measured, with the sidecar's exact widths, the line the whole text made where it first departs (20 paragraphs; 1 more is one the sidecar hands back, so it has no widths):

- The nearest starts 166,346 px along the shaped line, where a float32 step is 1/64 px. There the whole text accepted a line 0.01562 px wider than the box: one step.
- In all, 8 of these lines are wider than the box, by up to 0.211 px, and 12, all from 686,806 px on, are narrower: there the whole text moved to the next line a word that fits.
- Past 1,000,000 px all 11 lines are short, by 18.3 to 65.7 px of a 330 px box, and the whole text's height no longer equals the paragraphs' heights summed.

A `Text` holding a long document therefore wraps a paragraph one way, and the same paragraph shown on its own another, and the error grows with the length of the document. Drawn line by line, every paragraph is laid out from its own start, and its breaks do not depend on what precedes it.

7 · Costs

7.1 · Streaming

Exactness. 86 answers (215,423 characters) were appended token by token; tokens are a word's leading spaces with up to four of its characters. At every one of the 57,699 steps the line starts and the height equal Flutter's lay-out of the text so far; Flutter decided 84 steps.

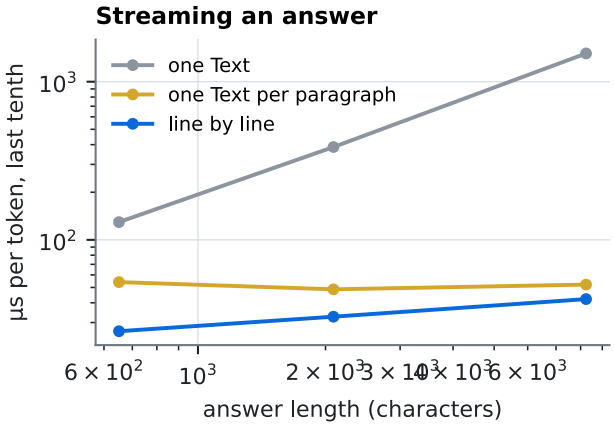


Figure 3. Cost of a token in the last tenth of an answer, by answer length (16 px, 330 px wide).

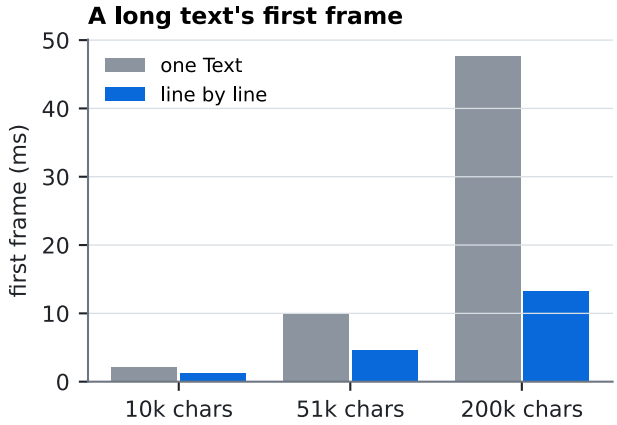


Figure 4. First frame of a long text: one `Text`, or the sidecar's breaks and the 44 lines of an 800 px viewport.

Answers	Paragraph	One Text	One Text per paragraph	Line by line	vs one Text	vs per paragraph
651 characters	152	75 (129)	36 (54)	26 (26)	2.9×	1.4×
2,088 characters	154	208 (386)	39 (49)	29 (33)	7.2×	1.3×
8,241 characters	169	797 (1,509)	46 (52)	36 (42)	22.3×	1.3×
One paragraph of 2,449	2,449	246 (456)	249 (461)	31 (34)	7.9×	8.0×

Table 5. Streaming, μs per token over the whole answer (in brackets: in its last tenth). "Paragraph": mean characters between blank lines. "One Text per paragraph" is how a Markdown view builds an answer, with finished paragraphs recorded once. Speed-ups compare whole streams.

One `Text` lays out everything streamed so far at every token, so its cost per token grows with the answer. One `Text` per paragraph bounds the cost by the current paragraph, and with this corpus's short paragraphs it is within 1.3× of line by line. Line by line bounds it by the last two lines: it is the only one of the three whose cost does not grow with a long paragraph.

7.2 · Long text

Text	One Text	Line by line	of which breaks	Speed-up	Jump to the middle: one Text	line by line

10,437 characters	2.12	1.24	0.33	1.7×	0.19	0.93
51,110 characters	9.99	4.56	3.66	2.2×	0.86	0.93
200,133 characters	47.70	13.31	12.33	3.6×	3.56	1.18

Table 6. Long text, ms. First frame: layout and the paint of the top of the text. "Of which breaks": the sidecar's breaks and heights for the whole text, including Flutter's layout of the paragraphs handed back. Jump: one `Text` paints its laid-out paragraph again, moved; line by line lays out the 44 lines that come into view.

The first frame gains less than streaming, because the breaks themselves are a pass over the whole text and the paragraphs handed back are laid out in full. A jump is cheaper for one `Text` on short texts, which has nothing left to lay out; at 200,133 characters painting all its lines again costs more than laying out a screen of new ones. A real list would keep painted lines in a layer; neither number includes rasterization.

8 · Related work

Lazy text layout is common, with estimated sizes: TextKit 2 lays out the viewport and estimates the rest, and Chrome's `content-visibility` skips layout of off-screen content and uses a placeholder size (`research/ENGINE.md`, §4–5). Line by line is lazy with exact sizes, because the breaks and heights come from a certified model rather than an estimate. Android's Minikin measures text per word through a cache and splits words at spaces, so it never shapes across a line end; SkParagraph shapes the paragraph as one line and never reshapes at line ends (§2 of the same note). Streaming chat interfaces commonly render Markdown, one widget per block, which is the per-paragraph baseline of §7.1.

9 · Limitations

- **One font, one direction.** Roboto Regular, left to right. Text the sidecar does not certify (other fonts as fallback, complex scripts, emoji sequences) is handed back, and drawn by Flutter as today.
- **Not a widget yet.** The measurements use `dart:ui` paragraphs directly. A widget would also need selection across lines, caret and hit testing, a single semantics node for screen readers, and justification (a one-line paragraph is its own last line, so `TextAlign.justify` does not stretch it); none is implemented. Ellipsis is not covered.
- **UI thread only.** Raster time, GPU memory and memory for many small paragraphs are not measured.
- **No phone.** Android results come from an emulator; timings are from the Mac.
- **Corpora.** Machine-generated from local files; streaming answers are corpus paragraphs, not model output, and tokens are approximated.
- **Float rounding is engine behaviour.** §5–6 describe SkParagraph in Flutter 3.47.5; another version that keeps positions relative to each line would remove the differences, and with them §6's effect.

10 · Reproducibility

- `flutter_bench/lib/ideas.dart` and `lib/ideas_main.dart`: every experiment here (`IDEA=pixels`, `drift`, `long`, `streaming`); for example `flutter build macos --release -t lib/ideas_main.dart --dart-define=IDEA=streaming`, then run the app with `BF_OUT` set; `tool/run_android.sh` on the emulator;
- `flutter_bench/results/ideas_*.json`: the results; `ideas_machine.txt`: the machine;
- `paper/lines/make_numbers.py`: every number and figure, read from those files, with the claims of the text asserted; `paper/lines/build.py`: this page.